计算机学院实验报告

实验题目： 三种平面三角形点定位算法		学号：202200130077
日期：2024. 4. 2	班级：智能	姓名：薛润泽
Email：		

实验目的：

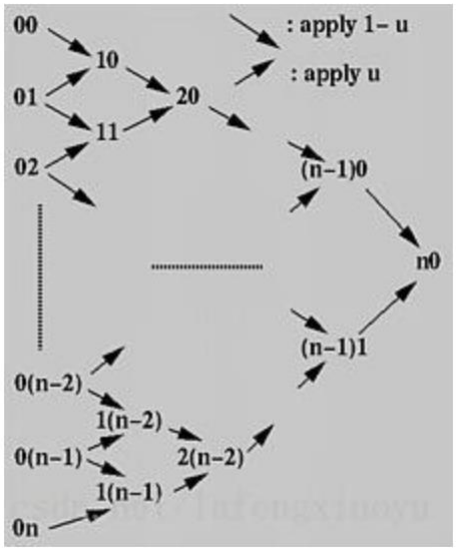
1、实现 de Casteljau 算法来返回 Bézier 曲线上对应点的坐标；
2、实现基于伯恩施坦多项式的算法返回 Bézier 曲线上对应点的坐标

实验环境介绍：

Ubuntu

解决问题的主要思路：

1、de Casteljau 算法可以是一个递推算法，每相邻两个点进行一次计算，得到一个新点的数值，递推直到最后只剩下一个点。
如图：



2、伯恩施坦多项式是一个二项展开式，描述了有 n 个点，且新点位置占原来连线为 p 时，最后得到点的计算式的系数项。

$$B_n(f,x) = B_n(f) = \sum_{k=0}^{n} f(\frac{k}{n}) C_n^K x^K (1-x)^{n-k},\ (x \in [0,1])$$

2、实现过程

recursive_bezier（使用多项式）和 recursive_bezier2（使用递推式）都用来计算当 p 固定时，原来 n 个点产生的新点。

bezier（多项式计算）和 bezier2（递推式计算）用来实际画图。

实验步骤与实验结果：

代码实现：

```
#include <chrono>
#include <iostream>
#include <opencv2/opencv.hpp>
#include <cmath>

long long factorial[50];
int To_size;
void calcfactorial(int n){
    factorial[0] = 1;
    for(int i = 1; i <= n; i++){
        factorial[i] = i*factorial[i-1];
    }
}
std::vector<cv::Point2f> control_points;

void mouse_handler(int event, int x, int y, int flags, void
*userdata)
{
    if (event == cv::EVENT_LBUTTONDOWN && control_points.size() <
To_size)
    {
        std::cout << "Left button of the mouse is clicked - position
(" << x << ", "
        << y << ")" << '\n';
        control_points.emplace_back(x, y);
    }
}
```

```
void naive_bezier(const std::vector<cv::Point2f> &points, cv::Mat
&window)
{
    auto &p_0 = points[0];
    auto &p_1 = points[1];
    auto &p_2 = points[2];
    auto &p_3 = points[3];

    for (double t = 0.0; t <= 1.0; t += 0.001)
    {
        auto point = std::pow(1 - t, 3) * p_0 + 3 * t * std::pow(1 -
t, 2) * p_1 +
                3 * std::pow(t, 2) * (1 - t) * p_2 + std::pow(t, 3)
* p_3;

        window.at<cv::Vec3b>(point.y, point.x)[2] = 255;
    }
}

cv::Point2f recursive_bezier(const std::vector<cv::Point2f>
&control_points, float t)
{   calcfactorial(control_points.size());
    // TODO: Implement de Casteljau's
    cv::Point2f pv(0,0);
    int n = control_points.size() - 1;
    for(int i = 0; i <= n; i++){
        pv += control_points[i] * std::pow(t, i) * std::pow(1 - t,
n-i)*(1.0 * factorial[n] / (factorial[i] * factorial[n - i]));
    }
    // return cv::Point2f();
    return pv;
}

cv::Point2f recursive_bezier2(const std::vector<cv::Point2f>
control_points2, float t)
{
    std::vector<cv::Point2f> control_points = control_points2;
    //    calcfactorial(control_points.size());
    // TODO: Implement de Casteljau's
    int n = control_points.size() - 1;
    while(n > 0){
        for(int i = 1; i <= n; i++){
```

```
            control_points[i-1] = t*control_points[i-1] + (1-
t)*control_points[i];
        }
        n--;
    }
    // return cv::Point2f();
    return control_points[0];
}



void bezier(const std::vector<cv::Point2f> &control_points, cv::Mat
&window)
{
    // TODO: Iterate through all t = 0 to t = 1 with small steps,
and call de Casteljau's
    // recursive Bezier algorithm.
    for(float i = 0; i <= 1; i += 0.001){
        auto point = recursive_bezier(control_points, i);
        window.at<cv::Vec3b>(point.y, point.x)[1] = 255;
    }
}

void bezier2(const std::vector<cv::Point2f> &control_points, cv::Mat
&window)
{
    // TODO: Iterate through all t = 0 to t = 1 with small steps,
and call de Casteljau's
    // recursive Bezier algorithm.
    for(float i = 0; i <= 1; i += 0.001){
        auto point = recursive_bezier2(control_points, i);
        window.at<cv::Vec3b>(point.y, point.x)[1] = 255;
    }

}
int main()
{
    cv::Mat window = cv::Mat(700, 700, CV_8UC3, cv::Scalar(0));
    cv::cvtColor(window, window, cv::COLOR_BGR2RGB);
    cv::namedWindow("Bezier Curve", cv::WINDOW_AUTOSIZE);
    cv::setMouseCallback("Bezier Curve", mouse_handler, nullptr);

    int key = -1;
    printf("input your to_size\n");
```

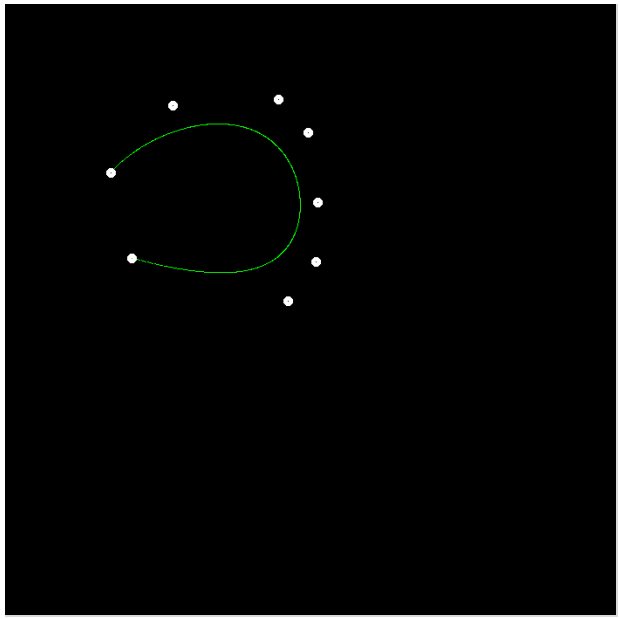
```
    scanf("%d",&To_size);
    while (key != 27)
    {
        for (auto &point : control_points)
        {
            cv::circle(window, point, 3, {255, 255, 255}, 3);
        }

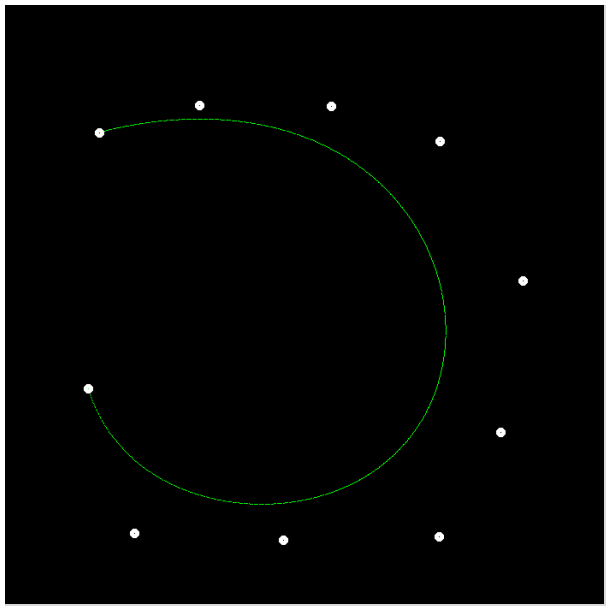
        if (control_points.size() == To_size)
        {
            // naive_bezier(control_points, window);
            // bezier(control_points, window);
            bezier2(control_points, window);
            cv::imshow("Bezier Curve", window);
            cv::imwrite("my_bezier_curve.png", window);
            key = cv::waitKey(0);
            return 0;
        }

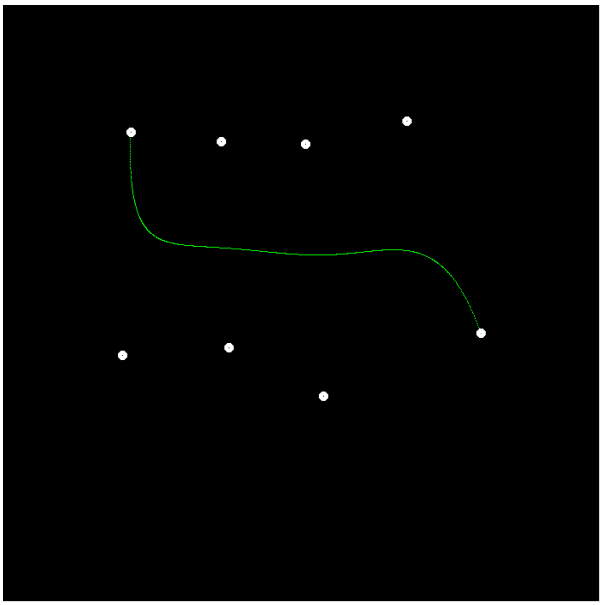
        cv::imshow("Bezier Curve", window);
        key = cv::waitKey(20);
    }

return 0;
}
```

实现效果如下：







实验中存在的问题及解决：

1、伯恩斯坦多项式中的组合数要先计算出相应阶乘，从而 O（1）计算组合数

2、de Casteljau 算法不要忘记多次迭代，直到只剩下一个点

3、实验原先给出的代码只能画 4 个点，要想画多个点，除了重新定义函数外，还要额外进行画图修改。